What Does PI-GNN Contribute to Graph Coloring?

Disentangling Optimization, Noise, and Discrete Repair

Machine Learning with Graphs — Project Report | 15 October 2026

Abstract

Physics-inspired graph neural networks (PI-GNNs) produce color probabilities for graph vertices, but a final coloring also depends on how those probabilities are decoded and repaired. We ask which stage actually improves solution quality. On 68 DIMACS graphs, with five runs per graph and one fewer color than the recorded chromatic number, we compare PI-GNN with direct optimization of vertex color scores and evaluate both under common local search and decoding. Only 42.1% of PI-GNN outputs admit no improving single-vertex recoloring after 100,000 steps. Directly optimized scores yield better initial colorings, whereas PI-GNN has a modest advantage after identical repair; adding sufficient noise to the direct optimizer reverses that advantage. Learned probabilities outperform deliberately disrupted inputs, but do not establish an overall advantage over a decoder given nearly uniform probabilities. These results show that PI-GNN learns useful information, while its benefit depends on the optimizer, decoder, and repair procedure used for comparison.

1 Introduction

In graph coloring, adjacent vertices should receive different colors. For example, vertices might represent exams and an edge might indicate that two exams cannot be scheduled simultaneously. When too few colors (time slots) are available, the practical goal becomes **minimizing conflicting edges**, not finding a conflict-free coloring.

PI-GNN (Schuetz et al., 2022) tackles coloring without labeled solutions. It learns a probability distribution over colors for every vertex by minimizing a differentiable approximation of the number of conflicts. Yet the probabilities are not themselves a coloring: a **decoder** chooses discrete colors, and **local search** may recolor vertices to remove further conflicts. A good final result can therefore reflect any of these stages.

The pipeline we investigate is:

Graph → optimization of vertex color probabilities → discrete decoding → optional single-vertex repair → conflict count.

We separate three questions. **RQ1 (endpoints):** Does PI-GNN finish at a coloring that cannot be improved by recoloring one vertex? **RQ2 (optimization):** Is the GNN better than simply optimizing an independent set of color scores for every vertex, especially after both outputs receive the same repair? **RQ3 (decoding):** How much useful information do the learned probabilities provide beyond what a graph-aware decoder can achieve by itself?

Our aim is to identify where quality comes from, rather than to claim a new coloring algorithm or state-of-the-art feasible coloring. The experiments deliberately use too few colors, so the optimal positive number of conflicts is generally unknown.

2 Related work

Schuetz et al. (2022) motivate the unsupervised Potts-loss formulation. Karalias and Loukas (2020) also distinguish learning probabilistic solutions from constructing discrete ones. Vanderbush and Weber (2026) already investigate differentiable coloring and a depth-zero direct-logit baseline: the mere GNN-versus-logits comparison is therefore not new. Our contribution is a controlled decomposition using matched starting outputs, independently tuned main optimizers, common repair, noise interventions, and decoder-input controls. We additionally compare with classical fixed-palette DSATUR (Brélaz, 1979) and TabuCol-style search (Hertz and de Werra, 1987).

3 Experimental design

3.1 What is optimized?

For an undirected graph $G = (V, E)$, suppose we have k colors. The model outputs a probability vector p_v for each vertex, obtained by applying softmax to its color scores (logits). We distinguish the **soft objective** from the actual **hard conflict count**:

$$L(P) = \sum_{\{u,v\}\in E} p_u^\top p_v,$$
$$C(c) = \sum_{\{u,v\}\in E} \mathbf{1}[c_u = c_v].$$

The soft loss L equals the expected number of conflicts if vertices independently sample colors from their distributions. **Argmax decoding** instead gives each vertex its most probable color; $C(c)$ counts the conflicts that remain. A lower value is better.

We compare two ways to optimize L. **PI-GNN** is a two-layer mean-aggregation GraphSAGE-style network (PI-SAGE) with trainable vertex embeddings, dropout, and AdamW; its output scores depend on shared neural parameters and graph-neighbor aggregation. **Free logits** remove that network and optimize a separate color-score vector for every vertex. Free logits still see the graph through L, but they do not perform message passing. To make their initial conditions comparable, they start from PI-GNN's saved initial output scores. Every model is **optimized separately on each test graph**; this is not a cross-graph generalization study.

3.2 Graphs, repair, and comparisons

We evaluate 68 DIMACS graphs, with 11–1,000 vertices and five random seeds each (340 runs per method). For every graph, we use $k = \chi - 1$, where χ is its recorded proven chromatic number. Assuming the metadata is correct, a conflict-free result is impossible. We use the final output after **100,000 full-batch optimization steps**, not the best earlier checkpoint.

All methods can be followed by the same **strict descent** repair. For each vertex, it checks whether changing its color reduces the number of conflicts with its neighbors. It repeatedly makes a largest strictly improving move (with seeded tie-breaking) until no such move exists; it never accepts a neutral or worsening move. We distinguish **raw conflicts** (before repair) from **repaired conflicts** (after it).

The GNN, free logits, a node-wise MLP, and a fixed initial neural linearization are independently hyperparameter-tuned on 13 disjoint synthetic graphs using the same search budgets. Noise variants optimize logits perturbed by Gaussian noise of standard deviation $\sigma \in \{0.5, 1, 2\}$ but are evaluated without noise; these variants reuse the free-logit hyperparameters rather than receiving separate tuning. RQ3 holds trained outputs fixed and varies their decoding or probability inputs. Additional protocols are in the supplementary material.

3.3 How to read the numbers

For each graph, we average its five seeds before comparing two methods. A **win/tie/loss** count therefore refers to graphs, not individual runs. We treat differences below one conflict, *or* relative differences at most 5%, as practical ties. The tables also report the median paired relative difference,

$$d_G = \frac{\overline{C}_X(G) - \overline{C}_Y(G)}{\max(\overline{C}_Y(G), 1)},$$

where X is the first method named. **Negative is better for the first method.** A median paired difference and an arithmetic mean can rank methods differently; practical ties are not statistical equivalence. Wilcoxon tests are descriptive on these heterogeneous graphs: RQ2 uses a 0.05/3 threshold for its co-primary comparisons, whereas RQ3 tests are unadjusted and its practical-tie rule followed an initial stricter inspection.

4 Results

4.1 RQ1: Confident predictions can still be improved

The endpoint question is simple: if a vertex has more neighbors of its current color than of another color, recoloring it immediately removes conflicts. We call a coloring **single-flip optimal** if *no* vertex has such an improving move. A strict minimum of the soft objective over unrestricted probability vectors would imply an even stronger property, but training a softmax-parameterized GNN need not reach that minimum (Supplementary Appendix A).

After 100,000 steps, only **143 of 340 PI-GNN outputs (42.1%)** are single-flip optimal. Seventeen graphs reach that condition in all five seeds; 30 never do. Across graphs, the median relative conflict reduction from subsequent repair falls from **94.9%** early in training to **10.5%** at the final checkpoint. Thus training improves the starts substantially but often leaves elementary work for repair.

High confidence does not explain this away. For the median graph, **99.66% of non-isolated vertices** have a maximum color probability of at least 0.99,

yet some confidently selected colors can still be locally improved. Softmax saturation can make gradients small even when a discrete recoloring helps; later training also changes some confident assignments, so confidence is not evidence that the coloring has frozen.

Repairing trained PI-GNN starts beats repairing random starts on **62 of 68 graphs** (five ties, one loss; median paired difference **−38.7%**). Nevertheless, against fixed-k DSATUR, repaired PI-GNN has only 16 wins, 28 ties, and 24 losses; a TabuCol reference finds fewer conflicts on 41 graphs under a non-equivalent runtime protocol. **Takeaway:** PI-GNN learns useful starts, but neither confidence nor single-flip optimality certifies a competitive final coloring.

4.2 RQ2: Repair reverses the baseline comparison; noise reverses it again

Free logits test whether the GNN architecture is needed to optimize the same soft objective. At 100,000 steps, plain free logits are single-flip optimal in all 340 runs and achieve better **raw** colorings on more graphs. But once both methods receive identical strict descent, PI-GNN wins on more graphs. This suggests that raw optimization quality and *usefulness as a repair starting point* are different properties; it does not prove which network mechanism causes that difference.

Variant	Stage	W/T/L	Median d_G
Plain logits	Raw	32/23/13	-8.58%
Plain logits	Repaired	17/25/26	$+5.99\%$
Noise $\sigma = 0.5$	Repaired	20/19/29	$+5.76\%$
Noise $\sigma = 1$	Repaired	35/30/3	-15.94%
Noise $\sigma = 2$	Repaired	44/21/3	-27.49%

Table 1: Free-logit variants versus PI-GNN on 68 graphs. W/T/L counts wins, practical ties, and losses for logits; negative median differences favor logits. All repaired logit outputs are unchanged by strict descent. Noise variants are secondary.

The reversal requires careful interpretation. Repaired PI-GNN has a better **median paired relative comparison**, not the lower arithmetic mean conflict count (114.9 for PI-GNN versus 108.6 for free logits). Its paired advantage also persists under the tested recorded-time matching and across ten extra repair tie seeds. These checks make a simple timing or tie-breaking explanation less likely, without equating total computational cost.

The stronger result is the noise intervention. With training noise $\sigma = 2$, free logits beat repaired PI-GNN on **44 graphs** and lose on just three. Likewise, removing dropout from PI-GNN eliminates its directional advantage over plain logits under the chosen rule. Noise and dropout act differently and were not independently retuned, so we do **not** infer a common mechanism. PI-GNN also beats the tested node MLP (28 wins, five losses) and its fixed initialization linearization (58 wins, no losses) after repair, although those comparisons do not isolate graph aggregation as the sole cause. **Takeaway:** the GNN's modest advantage over plain logits is conditional; a suitably perturbed simpler optimizer can surpass it.

4.3 RQ3: How much work is done by the decoder?

A decoder uses probability vectors to construct an actual coloring. Our **derandomized conditional-expectation decoder (D-CE)** fixes vertices sequentially, choosing the color that minimizes the expected remaining number of conflicts given already fixed colors and probabilities of unfixed neighbors. It uses the graph directly and guarantees $C(\mathrm{DCE}(P)) \leq L(P)$ in exact arithmetic. Both D-CE and ordinary argmax are followed by identical strict descent.

The essential control is **near-uniform probabilities**: when the input contains almost no learned preference, D-CE can still make graph-aware choices. We also disrupt learned inputs by independently scrambling each vertex's color coordinates, shuffling probabilities among vertices, or using an untrained network. Scrambling preserves vertex confidence but destroys coordination. These controls ask two different questions: *is the learned input better than misleading input?* and *is it better than giving the decoder virtually no learned guidance?*

First method/input vs. second	W/T/L	Median d_G
D-CE vs. argmax (trained)	22/46/0	-2.8%
Trained vs. near-uniform	27/28/13	0.0%
Trained vs. scrambled	48/20/0	-25.5%
Trained vs. vertex-shuffled	56/12/0	-29.9%
Trained vs. untrained	51/14/3	-22.2%

Table 2: D-CE decoding controls after identical repair, evaluated on frozen PI-GNN outputs. W/T/L favor the first method/input; negative differences favor it. The first three rows are primary comparisons.

D-CE improves argmax after repair, but only modestly in the median graph (**2.8%**). Trained probabilities consistently beat scrambled, shuffled, and untrained inputs, indicating useful learned alignment or coordination. Yet against near-uniform input the median advantage is **0.0%**, with 27 wins, 28 ties, and 13 losses (Wilcoxon $p = 0.227$). This is **not proof of equivalence**: trained inputs win on all 12 queen graphs, and other graph families sometimes prefer near-uniform input. Near-uniform D-CE is DSATUR-like, not exactly deterministic DSATUR.

A better decoder also does not erase differences between optimizers. Under common D-CE plus descent, noise-2 free logits still beat PI-GNN on **35 graphs**, tie on 29, and lose on four (median **−18.7%**). **Takeaway:** learned probabilities contain useful structure, but graph-aware decoding can supply much of the observed quality without it.

5 Discussion and limitations

Three distinctions matter. First, **confidence is not local optimality**: PI-GNN can assign nearly all vertices very high probabilities yet leave improving recolorings. Second, **raw quality is not repaired quality**: direct logits beat the raw neural output, while the ordering changes after identical local search. Third, **information relative to corrupted input is not information relative to an uninformative input**: the latter is a stronger test of how much learning adds to a graph-aware decoder.

The scope is deliberately narrow. We study one PI-SAGE implementation, optimization *per test graph*, final 100k-step outputs, heterogeneous DIMACS families, and $k = \chi - 1$; results do not establish behavior with feasible palettes or across GNN architectures. The relative paired summaries can disagree with arithmetic means, and several graph families have few instances. Hyperparameter searches use common protocols, not matched model capacity. Noise strengths and dropout removal are not independently retuned. Shared-hardware timing does not establish equal compute or end-to-end runtime. The RQ3 tie rule was adopted after an initial inspection, and statistical tests are descriptive, not claims of equivalence or population-wide superiority.

A useful next control would pass *hardened* learned probabilities to D-CE, separating useful non-argmax probability mass from the sequential recoloring that D-CE itself performs. Independently tuning noisy logits and changing graph structure systematically would further test the conditional advantages observed here.

6 Conclusion

PI-GNN learns better-than-random starting colorings, but its confident outputs frequently need elementary repair. It has a modest paired repaired advantage over plain free logits, which disappears when the direct optimizer uses sufficient noise. Its learned probabilities retain useful coordination, though a graph-aware decoder with nearly uniform input remains competitive overall. Evaluating neural coloring therefore requires **matched optimization baselines, identical repair, and decoding controls** rather than a single final conflict score.

References

Brélaz, D. (1979). New methods to color the vertices of a graph. *Communications of the ACM*, 22(4), 251–256.

Hertz, A., and de Werra, D. (1987). Using tabu search techniques for graph coloring. *Computing*, 39, 345–351.

Karalias, N., and Loukas, A. (2020). Erdos Goes Neural: an unsupervised learning framework for combinatorial optimization on graphs. *NeurIPS*, 33, 6659–6672.

Schuetz, M. J. A., Brubaker, J. K., Zhu, Z., and Katzgraber, H. G. (2022). Graph coloring with physics-inspired graph neural networks. *Physical Review Research*, 4, 043131.

Vanderbush, K., and Weber, M. (2026). Neural algorithmic reasoning for approximate k-coloring with recursive warm starts. *arXiv:2601.05137v1*.